AI FAQ Assistant API

AnswerBot - AI-powered FAQ and customer support automation

Team ID	SWTID-2026-3976
Team Size	4
Team Leader	S. Danielraj
Team Members	Vignesh Kumar, Arul Edvish, Roydon Malachi Martin
Project Period	15 September 2026 - 29 September 2026

Project Description

The AI FAQ Assistant is a robust, secure and intelligent REST API platform designed to streamline FAQ creation, user authentication and AI-powered content management. It is built with Node.js and Express.js and uses MongoDB with Mongoose for data storage and validation. JWT provides stateless sessions and route protection, and bcrypt encrypts passwords. The backend supports automated FAQ generation and keyword search, and uses centralised error handling so that database details are never leaked.

Scenario-Based Case Study

Background: Priya is a customer support operations manager who handles a large backlog of repetitive customer questions but struggles to organise entries, classify topics, write precise summaries and track question trends.

Problem: Keeping documentation current is difficult; there is limited visibility into search matches and quick resolutions; drafting support answers by hand is slow; and there is no permission layer protecting private documentation from public edits.

Solution: A centralised, secure API that separates business logic from the front end and provides services for content creation, categories, persistence and AI text generation.

Usage: Authenticated creators register, request AI help, draft FAQs and update or remove their own records. Public consumers list published articles and search by keyword.

Outcome: Manual delays shrink, schema validation keeps data consistent, tokens ensure only authors modify their documents and indexed lookups make answers easy to find.

1. Software Requirements

- Operating System: Windows 10/11, macOS or Linux
- Node.js v18+ and npm v9+
- Express.js v4/v5
- MongoDB with Mongoose ODM
- Postman or ThunderClient for API testing
- Visual Studio Code or a similar editor

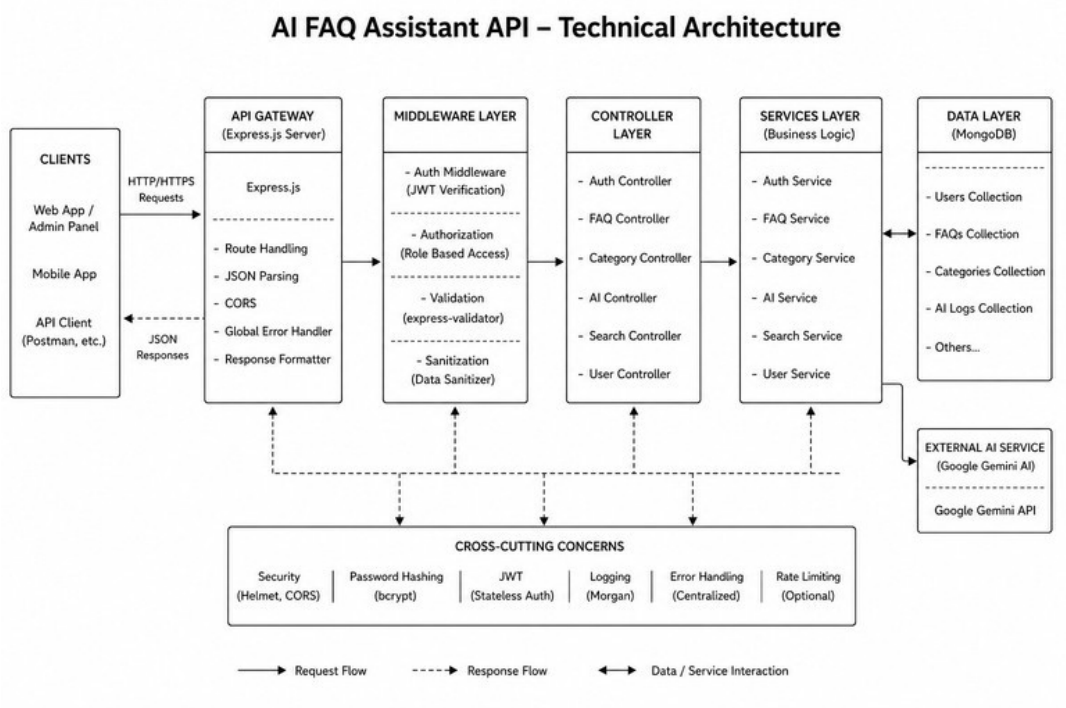
2. Hardware Requirements

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 1 GB of free disk space

Epic 1: Project Architecture

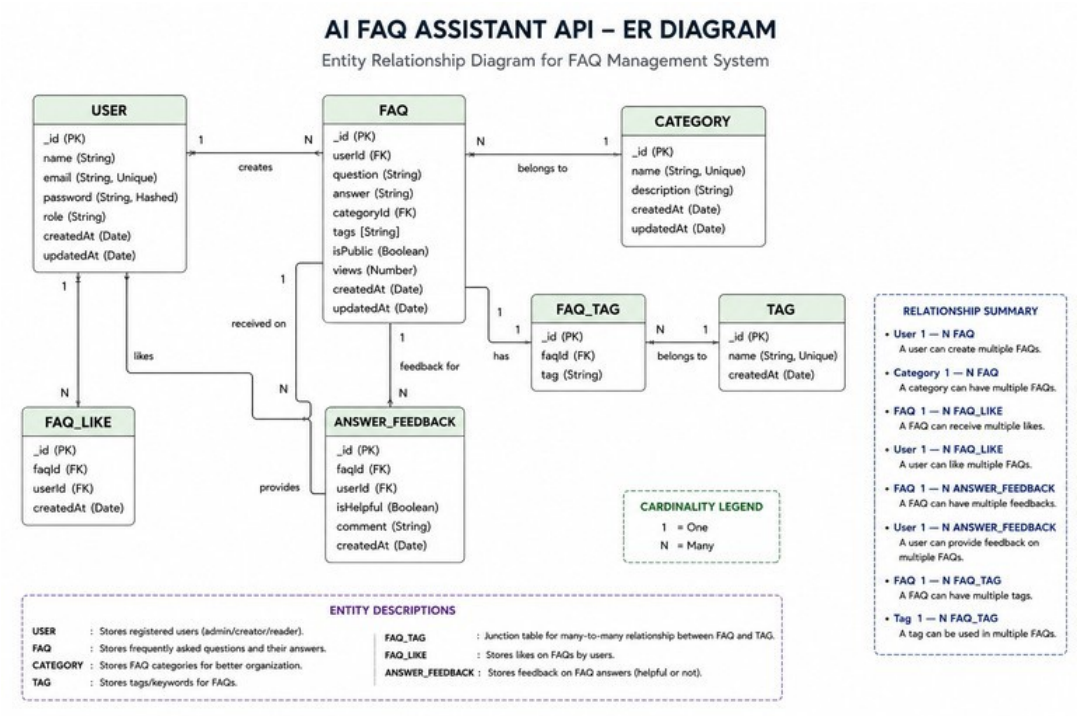
The application exposes structured endpoints handled by a layered architecture. Incoming requests are filtered by middleware before reaching the data services.

- **Interface:** API tools such as Postman query route paths and receive JSON.
- **Express Server Gateway:** listens on the port, maps HTTP methods, parses JSON and handles CORS.
- **Authentication Middleware:** reads the token from the header, validates it and authorises protected routes.
- **Services (logic modules):** isolate rules for user sessions and FAQ operations.
- **Database Interface Layer:** Mongoose turns application logic into MongoDB queries.
- **AI Flow:** the Google Gemini SDK handles question answering and topic-based FAQ generation.



Entity-Relationship (ER) Diagram

The database uses MongoDB collections connected by logical references. The implemented collections are User (name, email, password, role, timestamps) and FAQ (question, answer, category, createdBy, timestamps). The ER design also plans Category, Tag, FAQ_TAG, FAQ_LIKE and ANSWER_FEEDBACK entities. Key mappings: User 1-N FAQ, Category 1-N FAQ, FAQ 1-N FAQ_LIKE, FAQ 1-N ANSWER_FEEDBACK, FAQ N-N Tag through FAQ_TAG.



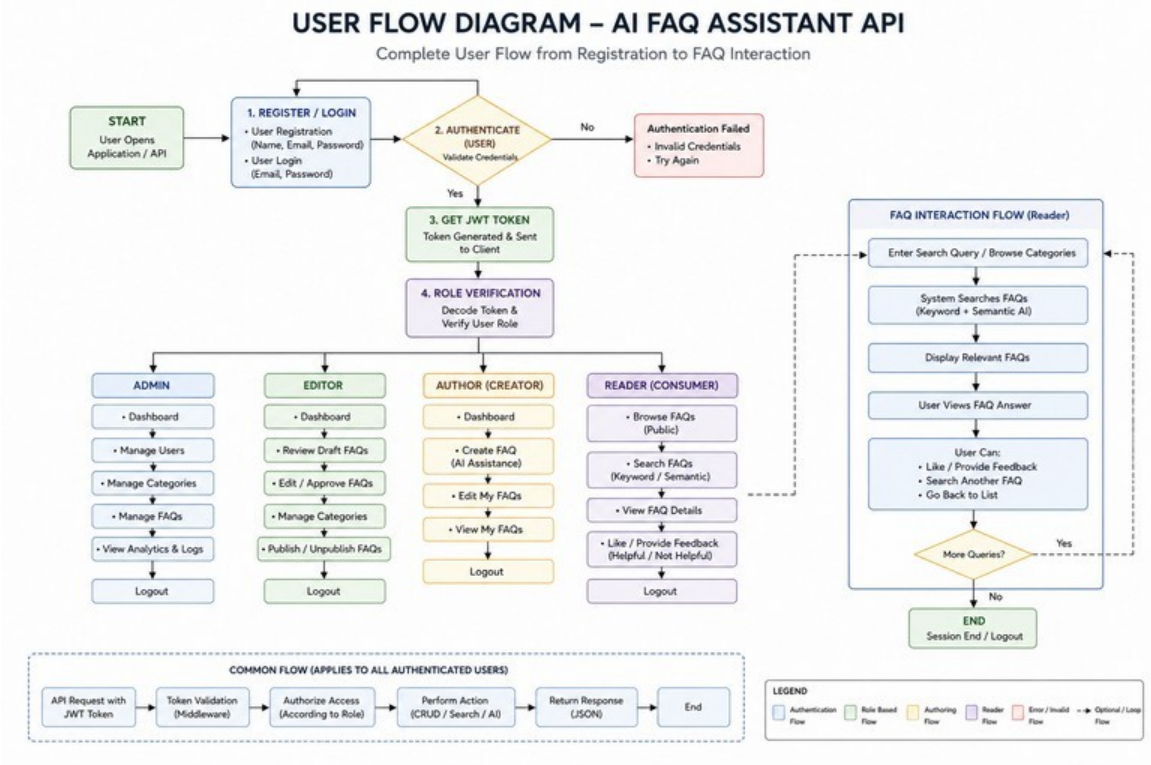
Key Features

- JWT authentication and authorisation for private endpoints
- AI FAQ generation and AI answers with Google Gemini
- Keyword-based FAQ search
- Complete CRUD operations for FAQs and category organisation
- bcrypt password hashing
- Centralised error handling and Mongoose schema validation
- Modular RESTful architecture on Express.js and MongoDB

Roles and Responsibilities

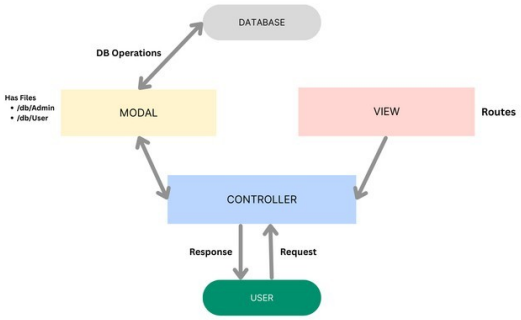
Role	Responsibilities
Admin (planned)	Manage users, view/update/delete any FAQ, monitor AI content and manage categories
Content Creator	Create FAQs, edit and delete own FAQs, generate FAQs using AI, organise FAQs under categories
Authenticated User	View and search FAQs, request AI-assisted answers, access own profile
Public User	View published FAQs and search; no access to protected endpoints

User Flow



MVC Architecture Pattern

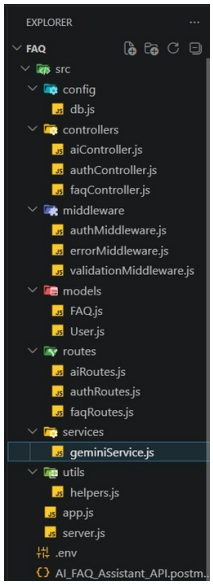
The source is organised around the Model-View-Controller pattern to decouple components. The **Model** layer holds Mongoose schemas that define fields, rules and defaults. The **Controller** layer validates requests, delegates work to models or services and returns standard JSON. In this headless service the **View** is the API routing layer that links request paths to controller handlers.



Epic 2: Project Setup and Configuration

Create the project folder with a `src` subfolder, open it in VS Code and run the following commands:

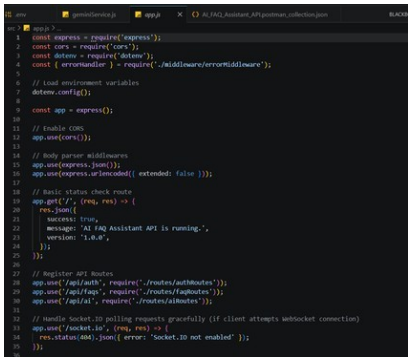
```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

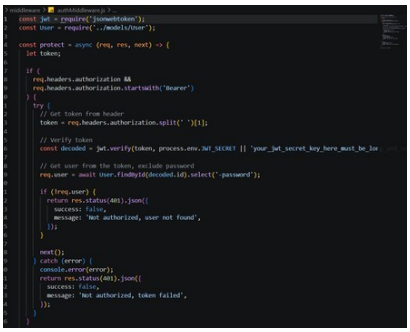


Epic 3: Backend Development

Core scripts: **src/server.js** loads environment variables, connects to the database and binds the listener port; **src/app.js** sets up Express, CORS, JSON parsing, routes and the error handler; **src/middleware/authMiddleware.js** verifies the Bearer token; **src/middleware/errorMiddleware.js** converts errors into clean responses.

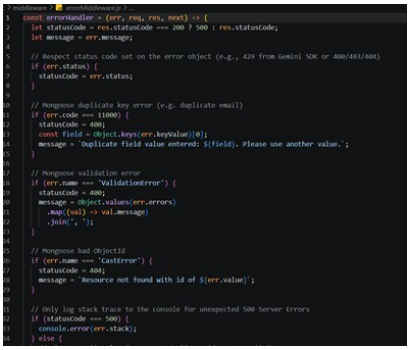


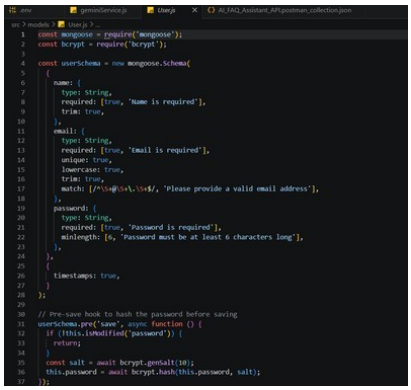


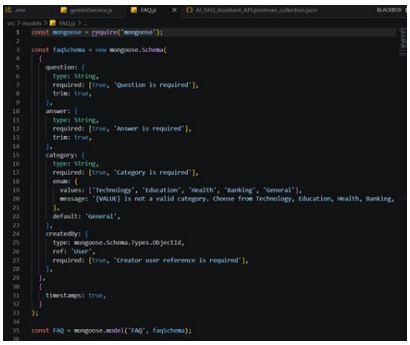


Epic 4: Database Configuration

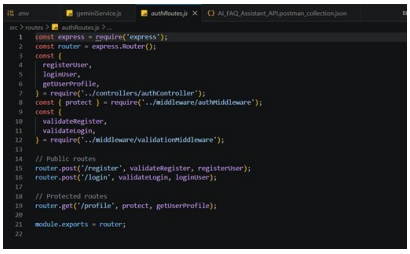
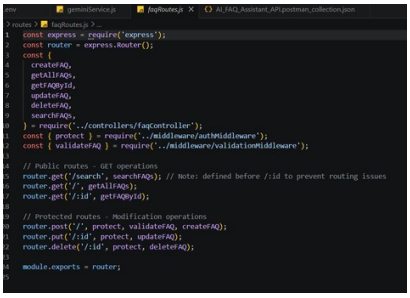
User.js enforces profile validations and hashes the password before saving. **FAQ.js** defines question, answer, category (enum) and a createdBy reference to the User.







API routers: **authRoutes.js** (register, login public; profile protected), **faqRoutes.js** (public GET, protected POST/PUT/DELETE) and **aiRoutes.js** (protected AI answer and generate-faq).

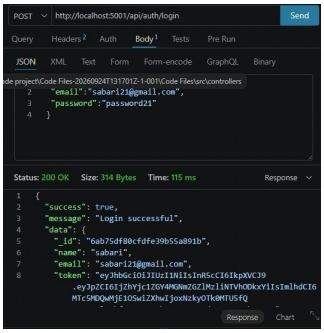
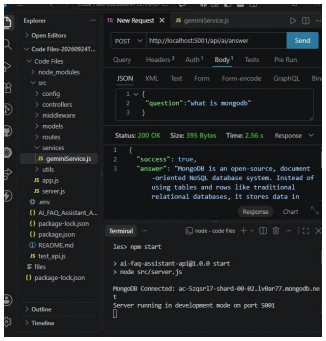
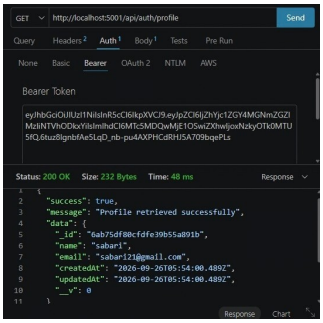
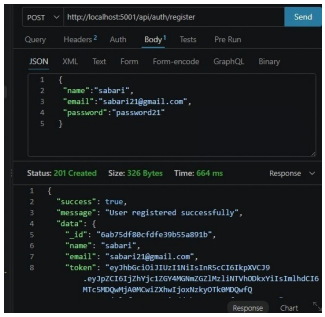

Epic 5: Project Execution - Running the API

Install Postman or ThunderClient, then start the server with `npm run dev`.

API Testing Specification

#	Route	Access	Payload
1	POST /api/auth/register	Public	{"name","email","password"}
2	POST /api/auth/login	Public	{"email","password"}
3	POST /api/faqs	Private (Bearer JWT)	{"question","answer","category"}
4	GET /api/faqs/search?q=configure	Public	-
5	POST /api/ai/generate-faq	Private (Bearer JWT)	{"topic"}

Postman results (register 201, login 200, profile 200, AI answer 200):

API TESTING VIDEO: [add link]
GITHUB LINK: [add link]